WARNING This material has been reproduced and communicated to you by or on behalf of the CQUniversity, Australia pursuant to Part IVA Division 4 of the Copyright Act 1968 (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice. Hassell, J 2017 *Learning PowerShell*. Boston, DeG Press. https://cqu.primo.exlibrisgroup.com/permalink/61CQU_INST/rpphdm/alma991001645923503441

Chapter 6
Filtering and Limiting

Welcome to Chapter 6 of *Learning PowerShell*. In Chapter 5, we learned all about objects, at least at a beginner's level. Objects are powerful differentiators that make PowerShell a rich and capable command-line environment. Understanding how to use objects and dig into their properties and methods unlocks the entire universe of PowerShell's abilities for you.

Most of the work you'll do with PowerShell, as I said in the previous chapter, involves looking at the properties of an object. But then you want to do something generally based on a specific property or two. For that, you need to be able to filter and limit. That's what this chapter is all about, PowerShell's built-in tools for filtering and limiting.

Let's dive in!

Introducing `Where-Object`

One of the most common uses of pipelining is to take the output of one cmdlet and filter it down into a certain subset of results; once you have filtered out the noise and you have your desired results, you then pipe that subresult set into another cmdlet to do some further magic.

This is where the `Where-Object` command comes in. `Where-Object` is one of the filtering mechanisms in PowerShell.

The Syntax for `Where-Object`

Now, the formatting of `Where-Object` gets a little funky, so stay with me while I show it to you. First off, you need to know some notations:

- `{` and `}` are curly brackets (in the same area as your backslash key, at least on US keyboards) and they denote logic. When you are making comparisons or setting up criteria for filtering, you enclose the criteria in curly brackets. It's kind of like the = sign in Excel that begins a formula, as in `=sum`, `=count`, `=average`, and so on.
- `$_` is shorthand for the current object. It's actually a little more complicated than that, but consider it to be the current object for the purposes of this chapter. Think of this in your head as whatever it is that you are working on. For example, the stuff (actually, the .NET object—remember from the last

Copyright © 2017. DEG Press. All rights reserved.

DOI 10.1515/9781501506673-006

Hassell, Jonathan. Learning PowerShell, DEG Press, 2017. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/cqu/detail.action?docID=4947050.
Created from cqu on 2022-07-19 04:40:49.

chapter that all PowerShell output is a .NET object) you get from the `Get-Process` cmdlet can be referred to in the pipeline next as `$_`.

- `-lt` is less than (i.e., 3 < 4, 3 is less than 4).
- `-le` is less than or equal to.
- `-gt` is greater than (i.e., 5 > 4, 5 is greater than 4).
- `-ge` is greater than or equal to.
- `-ne` is not equal to (i.e., `1 -ne 0`, 1 is not equal to 0).
- `-like` is how you use text phrases for match words, phrases, and other text-based patterns.

That might feel a little down in the weeds, but the good news is that this syntax, this way of formatting, is found all over PowerShell. Learn it once and you'll be good to go and can use it everywhere.

Using the `Where-Object` Goods

To use the `Where-Object` cmdlet, you simply type in `Where-Object` and then a left curly brace, and then you start adding properties to compare with. You finish it off with a right curly brace so PowerShell knows that is the end of your query.

We spent last chapter talking about objects and how objects have members that are called properties, which are characteristics and descriptors about those objects. Now is a great time to use properties in a simple example about how `Where-Object` is really interesting.

Let's consider the `Get-Process` cmdlet, which, as you might expect, gets a list of the processes currently running on any given system. Give it a shot now on your test system; you do not need to include any parameters with it.

```
Get-Process
```

That output is shown in Figure 6.1.

Copyright © 2017. DEG Press. All rights reserved.

Hassell, Jonathan. Learning PowerShell, DEG Press, 2017. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/cqu/detail.action?docID=4947050.
Created from cqu on 2022-07-19 04:40:49.

```
Windows PowerShell
Copyright (C) 2014 Microsoft Corporation. All rights reserved.

PS C:\Users\jhassell> Get-Process

Handles  NPM(K)    PM(K)      WS(K) VM(M)   CPU(s)     Id ProcessName
-------  ------    -----      ----- -----   ------     -- -----------
    301      16     3224       3528   101     0.31   4272 AdobeARM
    190      12     2196       4240    94            2216 atieclxx
    129       7     1028       1260    31             836 atiesrxx
    164      11     6900       9876    40     0.23   2032 audiodg
    708      36    31080      65752   296     0.59   5212 camera
   3280      94   122052       1528   963    94.64   1520 CCC
     68       8     1896       5820    58     0.03   3692 conhost
    294      12     1828       1912    48             432 csrss
    393      44     4124      15136   443             536 csrss
     66       6      960       2108    17            2156 dasHost
   1539     117   162080     169120   446 9,458.38   2128 Dropbox
    392      48    32872      41764   538             896 dwm
   2508     139   112608     178600   749   179.41    436 explorer
      0       0        0          4     0               0 Idle
    800      57    22176      53844   313    47.56   2356 iexplore
    781     196   219872     250424   536   221.22   5168 iexplore
    874     702   651396     688292  1064 2,447.05   5180 iexplore
    150      10     1660       2124    46             440 igfxCUIService
    175      13     3360       4820   100     0.14   3164 igfxEM
   1284      27     9404      14044    50             584 lsass
    218      13     3348       9132   100     0.64   3736 mobsync
    360      36    27504       2892   602     1.08   4536 MOM
    600      61   110268     129904   461            1660 MsMpEng
   1146      33    84304     104436   260     2.80   4548 mstsc
    267      20    13880       4772    59            2152 NisSrv
     96       9     1488       7664    97     0.38   6020 notepad
    453      23    61380      62600   610     0.34   4628 powershell
    175      24    32196      11232   518            2472 PresentationFontCache
    821      77    55568      63456   650            3472 SearchIndexer
    255      11     5532       8260    35             576 services
    490      20     8892       9272   127    10.00   4564 SettingSyncHost
    661      31    27176      24628   212    45.92   3716 SkyDrive
   1766     193   122612     117868   608   971.78   4936 Skype
     44       2      276        332     4             312 smss
    422      27     5532       7464    81            1268 spoolsv
    493      20     7128      11344    62             664 svchost
    483      18     5804       7796    45             756 svchost
    818      28    19196      20484   103             880 svchost
    791      30    29992      36244   126             912 svchost
   2083     104    57612      65668   745             948 svchost
    686      37    14124      19996   117             980 svchost
    995      48    15924      17904  1420            1040 svchost
    480      38    19688      18756   109            1296 svchost
    389      18     6908      12528   118            1440 svchost
     54       6      696       1096    21            1468 svchost
     51       6      684       1084    21            1500 svchost
    145      11     1928       2400    42            1636 svchost
    353      20     5264      11204    83            3244 svchost
    954       0      128      10424    16               4 System
    344      25     6028      10772   496    10.73   2304 taskhostex
    136      11     2720       7460    62             764 TiWorker
     96       8     1576       4668    30            2136 TrustedInstaller
     76       8      796        608    44             528 wininit
    165       9     1704       2712    59             700 winlogon
    177      13     3316       7936    41            5240 WmiPrvSE
```

Figure 6.1: The `Get-Process` command.

Let's say we want to find all processes that used more than 1,000 seconds on all the processors in the system. This can indicate a long-running process, or a process that's gone out of control, or something else. We'll use `Where-Object` to

Copyright © 2017. DEG Press. All rights reserved.

Hassell, Jonathan. Learning PowerShell, DEG Press, 2017. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/cqu/detail.action?docID=4947050.
Created from cqu on 2022-07-19 04:40:49.

do the filtering, and `Get-Process` to get all the initial process information from which we'll do our filtering. So in the PowerShell window, we'd type:

```
Get-Process | Where-Object { $_.
```

Now what do we type next? This is a great time to recall and remember and use the tab-ahead feature of the PowerShell window. You'll remember from Chapter 1 of this very book you hold in your electronic hands right now that tab ahead lets you press the Tab key to cycle through all the possible valid entries for whatever you're typing. Right now, since we just entered the period, PowerShell is going to let us cycle through all the available property names that we'd like to use. This is an incredibly valuable feature if you don't know exactly what you're looking for. Additionally, if you kind of know what you're looking for, it'll help you pinpoint it.

Here, since we know we're after CPU times, we can type in `CP` and press Tab and we'll see the PowerShell window add in the `U` for us, completing the identification.

```
Get-Process | Where-Object { $_.CPU
```

Time to build the actual comparison logic. We want everything greater than or equal to (so we'd use the `ge` tag) 1,000 CPU seconds. That will look like this:

```
-ge 1000
```

So now, we have built:

```
Get-Process | Where-Object { $_.CPU -ge 1000
```

And since we have completed building our "where" clause, we just add the right curly brace to finalize things and finish off our comparison statement.

```
Get-Process | Where-Object { $_.CPU -ge 1000 }
```

Run that at your console window, and see what happens.

Copyright © 2017. DEG Press. All rights reserved.

Hassell, Jonathan. Learning PowerShell, DEG Press, 2017. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/cqu/detail.action?docID=4947050.
Created from cqu on 2022-07-19 04:40:49.

```
PS C:\Users\jhassell> Get-Process | Where-Object { $_.CPU -ge 1000 }

Handles  NPM(K)    PM(K)      WS(K) VM(M)   CPU(s)     Id ProcessName
-------  ------    -----      ----- -----   ------     -- -----------
   1521     108   161988     169076   445 9,474.30   2128 Dropbox
    834     698   649564     687364  1053 2,450.09   5180 iexplore
```

Figure 6.2: The `Get-Process | Where-Object { $_.CPU -ge 1000 }` command.

Hark! A list of the two processes that have CPU seconds over 1000. Guess who the culprit is here? Looks like Dropbox has been running a long time on this system, which is true, given it has been up without a reboot for several weeks and I use Dropbox constantly (and it runs in the background synchronizing files, too).

What's pretty cool is that within your comparison logic, you can use the `and` parameter to add another set of criteria. For instance, in the above example, we could add another criterion that says we want the list to be filtered not only on CPU time, as we discussed, but also on the number of handles. More specifically, we want all processes that meet the CPU time criteria and have handles greater than 1,000.

See if you can build that clause yourself now, without looking ahead. We're six chapters into this book, so you should have enough self-confidence to take a stab at it. Go for it! I'll go get a cup of coffee.

Finished? That clause would look like this:

```
Get-Process | Where-Object { $_.CPU -ge 1000 -and
$_.Handles -ge 1000 }
```

See how that works? You can keep going on and on and on, but obviously at some point your filter becomes really tight. But that's how you add criteria to `Where-Object` and how you construct the comparison clauses.

Getting Lists and Filtering on Them

In this section, we'll return to an old friend: `Get-ChildItem`.

As you know, at its core, `Get-ChildItem` exists to get information on all of the things stored within something. If that something is the parent, the things stored within it are child items, and thus the name of the cmdlet was born. For example, a folder contains files, so when we point `Get-ChildItem` at a folder, what do you think we'd get back? Let's take a look in Figure 6.2.

Copyright © 2017. DEG Press. All rights reserved.

Hassell, Jonathan. Learning PowerShell, DEG Press, 2017. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/cqu/detail.action?docID=4947050.
Created from cqu on 2022-07-19 04:40:49.

```
PS C:\Users\jhassell> Get-ChildItem c:\Windows

    Directory: C:\Windows

Mode                LastWriteTime     Length Name
----                -------------     ------ ----
d----         8/22/2013  11:36 AM            addins
d----         8/22/2013  11:36 AM            ADFS
d----        10/11/2015   1:06 AM            AppCompat
d----        10/10/2015   5:01 PM            apppatch
d----        11/20/2015   8:18 AM            AppReadiness
d-r--        10/15/2015   4:16 AM            assembly
d----         8/22/2013  11:36 AM            Boot
d----         8/22/2013  11:36 AM            Branding
d----        11/21/2014  11:17 AM            Camera
d----        10/14/2015   8:01 AM            CbsTemp
d----        10/10/2015   3:29 PM            CSC
d----         8/22/2013  11:36 AM            Cursors
d----        10/10/2015   4:07 PM            debug
d-r--         8/22/2013  11:36 AM            DesktopTileResources
d----         8/22/2013  11:36 AM            diagnostics
d----         8/22/2013  11:43 AM            DigitalLocker
d---s         8/22/2013  11:36 AM            Downloaded Program Files
d----        11/21/2014  11:16 AM            en-US
d----        11/21/2014  11:17 AM            FileManager
d-r-s        10/14/2015   8:01 AM            Fonts
d----         8/22/2013  11:36 AM            Globalization
d----        11/21/2014   3:00 AM            Help
d----        11/21/2014  11:17 AM            IME
d-r--        10/10/2015   5:01 PM            ImmersiveControlPanel
d----        11/13/2015   4:47 PM            Inf
d----         8/22/2013  11:36 AM            InputMethod
d----         8/22/2013  11:36 AM            L2Schemas
d----         8/22/2013  11:36 AM            LiveKernelReports
d----        10/17/2015   3:00 AM            Logs
d-r-s         8/22/2013  11:36 AM            Media
d----        11/21/2014  11:17 AM            MediaViewer
d----        11/20/2015   3:55 AM            Microsoft.NET
d----        10/10/2015   3:45 PM            Migration
d----         8/22/2013  11:36 AM            ModemLogs
d-r--         8/22/2013  11:36 AM            Offline Web Pages
d----        10/11/2015   8:46 AM            Panther
d----        10/10/2015   5:10 PM            PCHEALTH
d----         8/22/2013  11:36 AM            Performance
d----         8/22/2013  11:36 AM            PLA
d----        10/10/2015   5:01 PM            PolicyDefinitions
d----        11/20/2015   1:01 AM            Prefetch
d----         8/22/2013  11:36 AM            Registration
d----        10/16/2015   3:46 AM            rescache
d----         8/22/2013  11:36 AM            Resources
d----         8/22/2013  11:36 AM            SchCache
d----         8/22/2013  11:36 AM            schemas
d----        11/21/2014   3:25 AM            security
d----         8/22/2013  10:45 AM            ServiceProfiles
d----        11/21/2014  11:17 AM            servicing
d----         8/22/2013  10:45 AM            Setup
d----        10/10/2015   5:11 PM            ShellNew
d----        11/21/2014   3:25 AM            SKB
d----        10/10/2015   3:48 PM            SoftwareDistribution
d----         8/22/2013  11:36 AM            Speech
d----         8/22/2013  11:36 AM            System
d-r--        11/17/2015   4:02 PM            System32
d----         8/22/2013  11:36 AM            SystemResources
d----        11/13/2015   4:44 PM            SysWOW64
d----         8/22/2013  11:36 AM            TAPI
d----        10/11/2015   9:38 AM            Tasks
d----        11/20/2015   9:43 AM            Temp
d-r--        10/10/2015   5:01 PM            ToastData
d----         8/22/2013  11:36 AM            tracing
d----        11/21/2014  11:17 AM            twain_32
d----         8/22/2013  11:36 AM            vpnplugins
d----         8/22/2013  11:36 AM            Vss
d----         8/22/2013  11:36 AM            Web
d----        10/10/2015   5:01 PM            WinStore
d----        10/14/2015   8:01 AM            WinSxS
```

Figure 6.3: The `Get-ChildItem` command pointed at a target folder.

Copyright © 2017. DEG Press. All rights reserved.

Hassell, Jonathan. Learning PowerShell, DEG Press, 2017. ProQuest Ebook Central,
http://ebookcentral.proquest.com/lib/cqu/detail.action?docID=4947050.
Created from cqu on 2022-07-19 04:40:49.

But `Get-ChildItem` is also capable of going up through subdirectories or subfolders (or down through them, depending on your point of view) and getting information about the child items within the folders of the original child items. This is called *recursion*, and you can enable this parameter simply by including `-recurse` in your command. For example, try this on for size in your own copy of PowerShell:

```
Get-ChildItem c:\windows -recurse
```

On my brand-new Core i5 machine at 2.9 GHz with 32 GB of RAM and a solid-state hard drive, that command took a few minutes (!) to run. That's because it's enumerating every single file within the Windows folder, even if those files are deeper within subfolders under C:. With one command. It would take you a lot more mouse clicks to get through the Windows Explorer GUI to do the same thing.

Are you beginning to see why they call it *Power*Shell?

Filtering, in a Pragmatic Sense

So let us use our newfound power for good and not for amusement. Let's say we have a server. Its main hard drive or storage array or default volume, or whatever you want to call it, is filling up. You suspect that some of your users are storing giant video files on this volume, and you want to find out if this is true. How could we find out in PowerShell?

Let's assume we're looking for files with the `.mp4` extension, as that's probably the most commonly used file. (In reality, you could search for any extension you wish, but this is a pretty decent example.) We want to find the video files in `c:\users` because our users don't have access to any other part of the disk; that is the root of their home directories shared out through SMB and mapped through Group Policy. That's a pretty standard setup you get out of the box in most Windows deployments if you are using folder redirection.

So we tell PowerShell we want information about files in `C:\Users` in this way:

```
Get-ChildItem c:\users -recurse
```

Now how do we search for the extensions? Remember, `Get-ChildItem` is going to return all of the information about the files, which would be information overload for our purposes. Let's jog our memories. What command do we use to

Copyright © 2017. DEG Press. All rights reserved.

Hassell, Jonathan. Learning PowerShell, DEG Press, 2017. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/cqu/detail.action?docID=4947050.
Created from cqu on 2022-07-19 04:40:49.

filter out information? Oh yeah, `Where-Object`. And how do we get things to `Where-Object`? Of course: the pipeline!

```
Get-ChildItem c:\users -recurse | Where-Object
```

Ah, we need the comparison logic now. We'll add the left curly brace (`{`) to begin the comparison expression, include the "that thing" notation (`$_`), and the period to access the properties of the .NET object that results from the `Get-ChildItem` cmdlet.

```
Get-ChildItem c:\users -recurse | Where-Object { $_.
```

Hmm. We're looking for extensions, so I'll type in `Ext` and see what comes up by tabbing through the options.

```
Get-ChildItem c:\users -recurse | Where-Object { $_.Ex-
tension
```

`-Extension` is one of them! There we go. So now I want the extension to equal `.mp4`, so I'll use the `eq` comparison (remember the list above) and then enclose `.mp4` in quotes because, well, take my word for it for now: when you're searching for text and not numbers, enclose the text in quotes.

```
Get-ChildItem c:\users -recurse | Where-Object {
$_.Extension -eq ".mp4"
```

Now just add the right curly brace and we are off to the races.

```
Get-ChildItem c:\users -recurse | Where-Object {
$_.Extension -eq ".mp4" }
```

Figure 6.4 shows what I get back.

Copyright © 2017. DEG Press. All rights reserved.

Hassell, Jonathan. Learning PowerShell, DEG Press, 2017. ProQuest Ebook Central,
http://ebookcentral.proquest.com/lib/cqu/detail.action?docID=4947050.
Created from cqu on 2022-07-19 04:40:49.

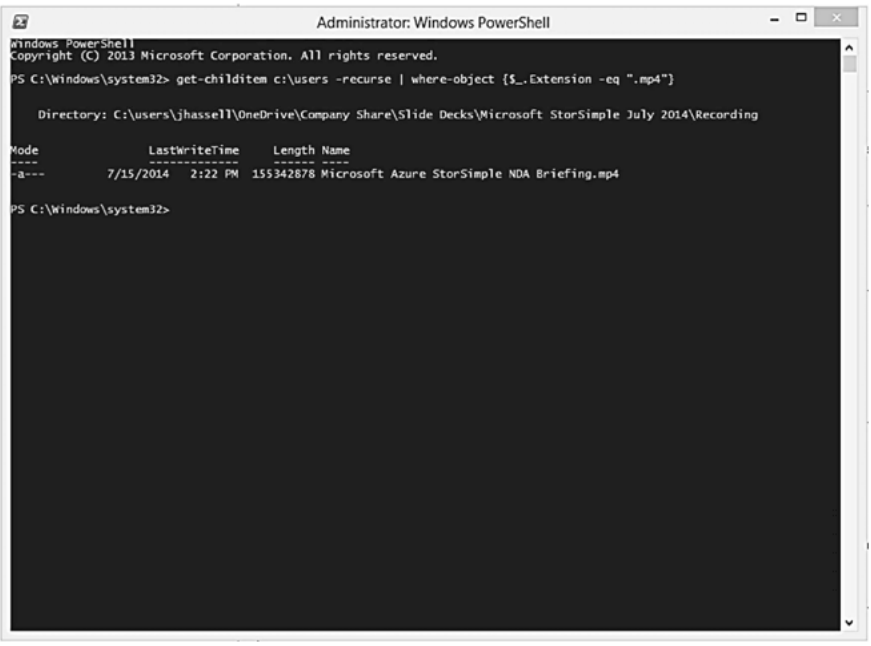

Figure 6.4: Searching for video and audio files in a target folder.

Voila! Now I don't have many video files on this system—just one—but if I ran this on my server, it would come back with quite a few. And other extensions would of course return different results. But now you know how to figure this sort of file management out with PowerShell and not with a billion right-clicks and shift-clicks inside the GUI.

Now some of you might get an error. This is probably because you are not running your Windows PowerShell instance as an administrator. Recall that we talked about this in Chapter 1; this is one of those instances when you are doing something that requires you to elevate yourself to an administrator account to carry out your intended action. Essentially, the reason why you need to run as an administrator is that when you search `C:\Users`, PowerShell can access only the parts of C: that your current user account has permission to access. By running as an administrator, you can generally search all of `C:\Users`.

Copyright © 2017. DEG Press. All rights reserved.

Hassell, Jonathan. Learning PowerShell, DEG Press, 2017. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/cqu/detail.action?docID=4947050.
Created from cqu on 2022-07-19 04:40:49.

Another Example of Objects and Filtering

Let's use another example, if for no other reason than to really hit the points home. Hypothetically, let's say you have been infected with Cryptolocker on one of your business' machines. This is a nasty ransomware bug; it is malware that silently encrypts the files it finds in a couple of places on your machine (My Documents and mapped drives being a couple of them). And then the bug makes you pay several hundred dollars in untraceable Bitcoin or Green Dot prepaid debit cards to get the key to decrypt them. You either pay up or you lose access to your files.

In our example, let's assume you were able to find the infection before it had the time to encrypt all your files. You immediately shut down the machine, so the encryption process stopped, but as part of your diagnosis of what happened, you need to figure out a list of all the files modified in the past day or so. Here we can again use `Get-ChildItem`, your tool of choice when you want to grab something out of a giant container of items—in this case the file system.

We know to start with `Get-ChildItem`, but how do we know what parameters to put along with it?

First, we can check out `Get-Help Get-ChildItem`, which will show us that the syntax starts off with `-Path`. We know that if we are concerned with potentially encrypted data at the mapped drive S: where shared documents are stored, we would use `-Path S:\` to establish where to look.

But what about subdirectories, subfolders, and any sort of nested structure we want to examine as well? From `Get-Help Get-ChildItem`, we also see the `-Recurse` parameter; as you now know, recursive checking means the program will start at the top and then "recurse," or walk down, the hierarchy of files until everything has been properly examined. We'll add that to the cmdlet as well.

That brings us to this partial cmdlet:

```
Get-ChildItem -Path S:\ -Recurse
```

You can actually run that, and PowerShell will spit out a list of every single file on the S: volume separated out by subdirectory. But we need to examine more about that huge list of files, so we will use the pipeline function to send that output into another cmdlet.

But what cmdlet helps us select a portion of a big set of data for further processing? Recall now that that function is the job of the `Where-Object` cmdlet.

So our cmdlet takes on further shape and body:

```
Get-ChildItem -Path S:\ -Recurse | Where-Object
```

Copyright © 2017. DEG Press. All rights reserved.

Hassell, Jonathan. Learning PowerShell, DEG Press, 2017. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/cqu/detail.action?docID=4947050.
Created from cqu on 2022-07-19 04:40:49.

Remember that we add in curly braces, and then within them we can use the `$_`, or as I like to affectionately call it, "that thing," to represent the output of a previous cmdlet being piped into a new cmdlet. Then, we add a period or dot and then the name of a property of that object represented by `$_`.

Here is what we have so far:

```
Get-ChildItem -Path S:\ -Recurse | Where-Object {$_.
```

But what is `Where-Object` going to filter? That's where we need to find out what the properties of `Get-ChildItem` are; we can use those properties to "tune the antenna," so to speak, of `Where-Object` so that it is filtering on the right criteria. Do you remember how to find the properties that can be filtered of any given object?

To find those properties, let us consult with `Get-Member`, which you learned about in Chapter 5. As a quick refresher, we know we need to use the command we're interested in and then pipe the output of that command to `Get-Member` so that it can properly examine the types of objects and their members that that given command puts out. So we'll use:

```
Get-ChildItem | Get-Member
```

And Figure 6.5 shows the results of that command on my system.

Copyright © 2017. DEG Press. All rights reserved.

Hassell, Jonathan. Learning PowerShell, DEG Press, 2017. ProQuest Ebook Central,
http://ebookcentral.proquest.com/lib/cqu/detail.action?docID=4947050.
Created from cqu on 2022-07-19 04:40:49.

```
Windows PowerShell
PS C:\Users\Jon> cd .\Desktop\
PS C:\Users\Jon\Desktop> Get-ChildItem | Get-Member


   TypeName: System.IO.FileInfo

Name                      MemberType     Definition
----                      ----------     ----------
Mode                      CodeProperty   System.String Mode{get=Mode;}
AppendText                Method         System.IO.StreamWriter AppendText()
CopyTo                    Method         System.IO.FileInfo CopyTo(string destFileName), System.IO.FileInfo CopyTo(s...
Create                    Method         System.IO.FileStream Create()
CreateObjRef              Method         System.Runtime.Remoting.ObjRef CreateObjRef(type requestedType)
CreateText                Method         System.IO.StreamWriter CreateText()
Decrypt                   Method         void Decrypt()
Delete                    Method         void Delete()
Encrypt                   Method         void Encrypt()
Equals                    Method         bool Equals(System.Object obj)
GetAccessControl          Method         System.Security.AccessControl.FileSecurity GetAccessControl(), System.Secur...
GetHashCode               Method         int GetHashCode()
GetLifetimeService        Method         System.Object GetLifetimeService()
GetObjectData             Method         void GetObjectData(System.Runtime.Serialization.SerializationInfo info, Sys...
GetType                   Method         type GetType()
InitializeLifetimeService Method         System.Object InitializeLifetimeService()
MoveTo                    Method         void MoveTo(string destFileName)
Open                      Method         System.IO.FileStream Open(System.IO.FileMode mode), System.IO.FileStream Op...
OpenRead                  Method         System.IO.FileStream OpenRead()
OpenText                  Method         System.IO.StreamReader OpenText()
OpenWrite                 Method         System.IO.FileStream OpenWrite()
Refresh                   Method         void Refresh()
Replace                   Method         System.IO.FileInfo Replace(string destinationFileName, string destinationBa...
SetAccessControl          Method         void SetAccessControl(System.Security.AccessControl.FileSecurity fileSecurity)
ToString                  Method         string ToString()
PSChildName               NoteProperty   string PSChildName=connectto365.ps1
PSDrive                   NoteProperty   PSDriveInfo PSDrive=C
PSIsContainer             NoteProperty   bool PSIsContainer=False
PSParentPath              NoteProperty   string PSParentPath=Microsoft.PowerShell.Core\FileSystem::C:\Users\Jon\Desktop
PSPath                    NoteProperty   string PSPath=Microsoft.PowerShell.Core\FileSystem::C:\Users\Jon\Desktop\co...
PSProvider                NoteProperty   ProviderInfo PSProvider=Microsoft.PowerShell.Core\FileSystem
Attributes                Property       System.IO.FileAttributes Attributes {get;set;}
CreationTime              Property       datetime CreationTime {get;set;}
CreationTimeUtc           Property       datetime CreationTimeUtc {get;set;}
Directory                 Property       System.IO.DirectoryInfo Directory {get;}
DirectoryName             Property       string DirectoryName {get;}
Exists                    Property       bool Exists {get;}
Extension                 Property       string Extension {get;}
FullName                  Property       string FullName {get;}
IsReadOnly                Property       bool IsReadOnly {get;set;}
LastAccessTime            Property       datetime LastAccessTime {get;set;}
LastAccessTimeUtc         Property       datetime LastAccessTimeUtc {get;set;}
LastWriteTime             Property       datetime LastWriteTime {get;set;}
LastWriteTimeUtc          Property       datetime LastWriteTimeUtc {get;set;}
Length                    Property       long Length {get;}
Name                      Property       string Name {get;}
BaseName                  ScriptProperty System.Object BaseName {get=if ($this.Extension.Length -gt 0){$this.Name.Re...
LinkType                  ScriptProperty System.Object LinkType {get=[Microsoft.PowerShell.Commands.InternalSymbolic...
Target                    ScriptProperty System.Object Target {get=[Microsoft.PowerShell.Commands.InternalSymbolicLi...
VersionInfo               ScriptProperty System.Object VersionInfo {get=[System.Diagnostics.FileVersionInfo]::GetVer...
```

Figure 6.5: Finding the properties of `Get-ChildItem` we can use.

Looking at that result, I can see two properties that might be interesting to us for completing our task: `LastWriteTime` and `LastWriteTimeUtc`. This is what we're looking for! We need the last time that a file was written to.

In this case, just to make things simple, we will use `LastWriteTime` rather than worrying about converting time zones to Greenwich Median Time, although you might have a specific purpose for doing so as you progress in your scripting capabilities.

So to put together our fuller picture, here is where we are:

```
Get-ChildItem -Path S:\ -Recurse | Where-Object {$_.LastWriteTime
```

Copyright © 2017. DEG Press. All rights reserved.

Hassell, Jonathan. Learning PowerShell, DEG Press, 2017. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/cqu/detail.action?docID=4947050.
Created from cqu on 2022-07-19 04:40:49.

So we have identified the last write time, but we obviously need to do something with that; we need to ask ourselves, in constructing this command, the question: "Where the last write time is *what*, exactly?" So we need a comparison operator.

We learned in the first section of this chapter that we can use `-lt` for "less than" and `-gt` for "greater than." So in order to figure out what was written in the last day or so, we can pick a date two days ago. In this example, today is May 14, 2015, so if I'm trying to figure out what files have been touched in the past 24 hours, I would want to know files where the last write time is greater than May 12, 2015.

We write this out in standard MM/DD/YYYY format and then enclose it in quotes, as it is considered a string. Then we will add the closing curly brace because our comparative clause is complete, and we have the following cmdlet built:

```
Get-ChildItem -Path S:\ -Recurse | Where-Object
{$_.LastWriteTime -gt "05/12/2015"}
```

Run that, and you will get a list of every file on the S: volume that has been written to on 5/12/2015 or after—exactly what we were looking for. And we did that by understanding that

- (a) the output of `Get-ChildItem` is an object, and
- (b) we can find the properties of the `Get-ChildItem` output object using `Get-Member` and use those properties to
- (c) pipe to `Where-Object` to find specific information about a subset of that output.

Further Exploration

There are all sorts of convenient ways to use objects and their properties and methods. With all output being an object, it means you can sort and filter all sorts of attributes and characteristics of whatever it is you are working on.

For instance, you can display information in a table format that eliminates all the other facts you have no interest in and laser-focuses on facts you are interested in. For example, let's look at what is available for `Get-Service`.

```
Get-Service | Get-Member
```

If I run that, I will see in the resulting table that `Status` is a property and `Start` and `Stop` are methods. So if I wanted to find out all the services on a machine

Copyright © 2017. DEG Press. All rights reserved.

Hassell, Jonathan. Learning PowerShell, DEG Press, 2017. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/cqu/detail.action?docID=4947050.
Created from cqu on 2022-07-19 04:40:49.

that were in the Stopped state, and start those services, I might want to build the following cmdlet:

```
Get-Service | Where-Object {$_.Status -eq "Stopped"} |
Start-Process
```

Or consider a different scenario with different software. What if I wanted to find all of the Exchange mailboxes that had been created in my lab Exchange environment and then delete those mailboxes because I am done with my experiment and want to restore my test deployment? First, I would want to see the properties available for the `Get-Mailbox` cmdlet, a core cmdlet of Exchange or Office 365:

```
Get-Mailbox | Get-Member
```

I would see, among dozens of other properties, the `WhenChanged` property. This might work, so I would test this out:

```
Get-Mailbox | Format-List name,WhenChanged
```

This gives me a list of mailboxes with the mailbox-friendly name and the value of the `WhenChanged` property. Looks like what I need, so I will modify the above cmdlet not to display a list, but to receive the output of `Get-Mailbox` into a `Where-Object` filter, where I will grab the `WhenChanged` output and pass, for deletion, only those that meet my comparison criteria via the pipeline to the `Remove-Mailbox` cmdlet. It ends up looking like this:

```
Get-Mailbox | Where-Object {$._WhenChanged -gt
"05/07/2015"} | Remove-Mailbox
```

Voila.

The Last Word

As I mentioned, my goal for this chapter of *Learning PowerShell* was to show you PowerShell's built-in tools for filtering and limiting. Now in the next chapter, we'll put objects, filtering, and limiting together and make some simple scripts.

Onward!

Copyright © 2017. DEG Press. All rights reserved.

Hassell, Jonathan. Learning PowerShell, DEG Press, 2017. ProQuest Ebook Central,
http://ebookcentral.proquest.com/lib/cqu/detail.action?docID=4947050.
Created from cqu on 2022-07-19 04:40:49.